No Conserved Program:
Reversibility and Homoiconicity in a Two-Loop Toffoli Machine

The moop authors*

October 2026

Abstract

Reversibility and homoiconicity are usually studied apart: the first as a property of the semantics and thermodynamics of computation, the second as a property of how a language represents its own code. We study them together in moop, a small prototype-based language designed to be homoiconic throughout, with reversibility reserved for its system-facing code. Every object's body is two counter-rotating circular tapes coupled by a pair of Toffoli gates, and user-facing code is kept as data in a conventional, irreversible layer. We show that this core is exactly a pair of nonlinear feedback shift registers coupled by one AND term, and we define homoiconicity structurally, as the absence of an autonomous program subsystem. The language's own history supplies a control: an early implementation broke the intended homoiconicity by leaving one tape read-only—a Harvard machine in miniature—until the wiring was corrected. An exhaustive census of all 2^{21} tape states of the 8/13-cell machine makes the difference measurable. The control conserves its program and splits into 442,990 orbits, none longer than 255 epochs; the homoiconic wiring has 8,938 orbits, every non-trivial one changes both tapes, and the longest visits 8,191 of the 8,192 possible programs. Across all 100 loop pairs with at most 22 cells, only 184 short orbits, covering 438 of 78 million states, restore one tape at every epoch boundary. We then state the synergy precisely: reversibility makes a state's whole history recoverable, and homoiconicity makes the program part of that history, so a program that keeps rewriting itself can still be recovered from any single state. Because a reversible core never merges states, every many-to-one notion of "answer" must come from a forgetful observation outside it—the boundary that moop's two-layer segregation draws. A transcript theorem, for a calculus of the layer crossings, shows that system information is lost only by pruning or by exporting it through observation; and causal pruning, the one internal erasure, provably never alters the past of the cells it keeps. moop thus occupies two cells of the reversibility–homoiconicity square on purpose: system-facing code sits in the causally closed cell, and user-facing code in the homoiconic but forgetful one, where answers live. Every measured number in the paper is reproduced by a single harness.

1 Introduction

Two properties of computing systems have been studied by largely disjoint communities. *Logical reversibility*—every step can be undone because no step destroys information—comes from the physics of computation: Landauer's observation that erasure has a thermodynamic cost [16], Bennett's reversible Turing machines and the technique of uncomputation [2, 4], and Toffoli and Fredkin's reversible gates and conservative logic [8, 26]. *Homoiconicity*—programs are data in the same medium the program manipulates—comes from language design, with Lisp as the canonical

*Draft. Source, harness, and data: `docs/paper/` in the moop repository.

example [19]; its architectural counterpart is the stored-program idea, in which code lives in writable memory alongside data [30].

Neither literature asks what each property does *for the other*. The design notes of moop, a small prototype-based language, make a specific claim about exactly that:

> *Neither property alone yields any of this: reversibility with an external program keeps the system irreversible; homoiconicity without reversibility is self-modification with amnesia.*

This paper turns that claim into definitions, theorems, and measurements. The design behind it is specific: moop is meant to be homoiconic everywhere and reversible only where it faces the system. Homoiconicity is *layered*—system code is data on the tapes, user code is data in ordinary memory and, through quotation, a value of the language—while reversibility belongs to the system-facing code alone. The two properties therefore meet in the system layer, and the user layer keeps homoiconicity without reversibility by choice. moop's objects have reversible bodies built on a single primitive—two counter-rotating circular tapes ("loops") coupled by a pair of Toffoli (CCNOT) gates—and a conventional, forgetful user layer sits beside them. The core is small enough to enumerate: every state of the 8/13-cell machine the interpreter uses can be visited.

Contributions.

1. A closed form for the core: two nonlinear feedback shift registers coupled by a single AND term (§4).
2. A structural definition of homoiconicity—no autonomous program subsystem—and an exhaustive orbit census comparing moop's wiring with its own first, Harvard-style wiring (§5).
3. The synergy as a set of precise statements: recoverable program history, no convergence of distinct programs, answers as observations, and recurrence (§6).
4. Causal pruning, the system layer's one internal erasure, with a rewind lemma and an account of what its metadata records (§7).
5. A transcript theorem for a calculus of the crossings between the two layers, showing that every other loss of system information is an export through observation (§8).
6. Consequences for the language surface, including a route to general functions as involutions whose forward direction needs only user-layer computation (§9).

Non-claims. We prove no universality result: the loops are finite and the wiring is fixed. We describe no hardware and make no efficiency claim. And moop programs do not execute *on* the core: the interpreter runs on a conventional host and uses the core for observation, deposits, and births. Section 9 discusses why this placement is consistent with the theorems.

2 Background

Reversible computing. A computation is logically reversible when its step function is injective, so that the previous state is determined by the current one. Landauer tied the erasure of a bit to a minimum dissipation of $kT \ln 2$ [16]; Bennett showed that any computation can be made reversible by keeping a history and later uncomputing it [2], and analysed the space–time cost of doing so [4]. Toffoli's controlled-controlled-NOT gate is universal for reversible Boolean circuits given constant ancillae [26]; without ancillae, NOT, CNOT and CCNOT on $n \geq 4$ wires generate exactly the even permutations, and odd ones need a single extra wire [23]. Bennett's resolution of Maxwell's demon locates the unavoidable cost not in measurement, which can be reversible, but in erasing the demon's memory [3]. Toffoli and Margolus built reversible cellular automata [27]; Morita surveys reversible machines of many kinds [21].

Reversible languages and the place of irreversibility. Janus [18, 31] is a reversible imperative language in which programs can be inverted syntactically; reversible programs compute exactly the injective functions [1]. James and Sabry treat irreversibility as a computational effect over a reversible core, with the creation and erasure of information as its two primitives [14]. Causal-consistent reversibility [6] tracks dependencies so that concurrent steps can be undone in a causally consistent order. Knill's QRAM model [15] describes a classical host that prepares, drives, and measures a reversible (unitary) coprocessor. Reversible memory management has been studied for functional languages [20].

Two senses of homoiconicity. *Representational* homoiconicity means programs are values of the language's own data structures (Lisp [19]; reflection in the sense of Smith [24]). *Architectural* homoiconicity is the stored-program concept: code lives in memory that execution can write. Its opposite is the Harvard organisation, where the program store is separate and read-only to execution. `moop` intends both, one per layer: the architectural kind on the tapes, the representational kind for user code. Circulating memory as such is old—EDVAC's mercury delay lines were loops [30]—so nothing here depends on the loops being novel.

Shift registers. A feedback shift register of length n computes $s_{t+n} = f(s_t, \ldots, s_{t+n-1})$; it is linear (an LFSR) when f is an XOR of taps and nonlinear (an NLFSR) otherwise [10]. Stream ciphers such as Trivium [5] and Grain [12] couple shift registers through AND terms, using state updates that are invertible.

3 The machine

Definition 1 (Core)**.** A *core* has two loops, $A \in \{0,1\}^a$ and $B \in \{0,1\}^b$ with $a, b \geq 2$, and two heads $h_A \in \mathbb{Z}_a$, $h_B \in \mathbb{Z}_b$, initially 0. One tick is *fire* followed by *rotate*:

$$\begin{aligned} \mathsf{F}: &\quad g \leftarrow A[h_A] \wedge B[h_B]; \quad A[h_A+1] \leftarrow A[h_A+1] \oplus g; \quad B[h_B-1] \leftarrow B[h_B-1] \oplus g, \\ \mathsf{R}: &\quad h_A \leftarrow h_A + 1; \quad h_B \leftarrow h_B - 1. \end{aligned}$$

A tick in which $g = 1$ is a *firing*. The *epoch* is $L = \operatorname{lcm}(a, b)$ ticks, after which both heads are back at their starting positions. The *epoch map* E sends a tape state (A, B), with heads at $(0, 0)$, to the tape state one epoch later.

Figure 1 shows one tick. The implementation is 111 lines of C (`src/tapeloop.c`).

Proposition 2 (Exact inverse)**.** *F is an involution and R is a permutation, so a tick $\mathsf{R} \circ \mathsf{F}$ is a bijection with inverse $\mathsf{F} \circ \mathsf{R}^{-1}$.*

Proof. Because $a, b \geq 2$, the targets $h_A + 1$ and $h_B - 1$ are distinct from the controls h_A and h_B, so F does not change g. Applying F twice XORs each target with the same g twice, which is the identity. R is a rotation of head positions. □

The bound $a, b \geq 2$ is necessary. With $a = 1$ the target on A is the control itself. If $B[h_B] = 1$, the state with $A = 1$ fires and steps to $A = 0$ with $B[h_B - 1]$ flipped; the state that already has $A = 0$ and $B[h_B - 1]$ flipped does not fire, and steps to the same place. The C function `moop_core_step_back` is $\mathsf{F} \circ \mathsf{R}^{-1}$—the forward sequence reversed, each part self-inverse—and a unit test round-trips a full epoch. This shape returns in §9.

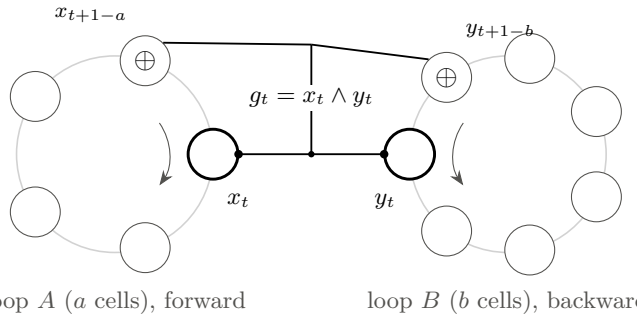


Figure 1: One tick of the core. The cells under the heads (thick) are the controls; their AND g_t is XORed into the next cell of *each* loop—the cell that rotates under the head next ($\oplus$). Labels give each cell's value in the sequence notation of Theorem 4. The original wiring had only the target on A, so B was never written.

The control: the original wiring. `moop` was meant to be homoiconic from the start, but the first implementation of its core (commit `2231ab2`) had a single CCNOT, targeting A only. Loop B was then never written: a read-only program driving a data loop—a Harvard machine in miniature. That implementation broke the intent. Commit `745ecae` corrected it by adding the symmetric target on B, and in the same change wrote the requirement down ("no loop may become a read-only program tape"). We keep the original wiring as a control throughout: the same gate, loops, and rotation, with and without architectural homoiconicity. We call it the *Harvard wiring*.

4 A closed form: two coupled shift registers

Write A_t, B_t for the loops after t ticks, and let

$$x_t = A_t[t \bmod a], \qquad y_t = B_t[-t \bmod b]$$

be the cells under the heads at tick t. Extend both sequences to negative indices with the initial tapes: $x_\tau = A_0[\tau \bmod a]$ for $-a < \tau < 0$ and $y_\tau = B_0[-\tau \bmod b]$ for $-b < \tau < 0$.

Lemma 3 (Window)**.** *For all $T \geq 0$ and $T - a < \tau \leq T$, $A_T[\tau \bmod a] = x_\tau$; likewise $B_T[-\tau \bmod b] = y_\tau$ for $T - b < \tau \leq T$. Each loop holds exactly the last a (respectively b) symbols of its head sequence.*

Proof. Cell i of A is under the head at ticks $t \equiv i$ and is a target at ticks $t \equiv i - 1 \pmod a$: it is written only on the tick immediately before it is read. Let $i = \tau \bmod a$. If $\tau \geq 0$, the cell was read at tick τ with value x_τ, and its next write happens at tick $\tau + a - 1$, which has not run yet because $\tau + a - 1 \geq T$. If $\tau < 0$, the cell's first write is at tick $\tau + a - 1 \geq T$, so it still holds $A_0[i] = x_\tau$. Loop B is identical with the head moving backward. □

Theorem 4 (Coupled NLFSRs)**.** *For all $t \geq 0$, with $g_t = x_t \wedge y_t$,*

$$x_{t+1} = x_{t+1-a} \oplus g_t, \qquad y_{t+1} = y_{t+1-b} \oplus g_t.$$

Proof. Tick t XORs g_t into $A[(t+1) \bmod a]$, which by Lemma 3 (with $T = t$, $\tau = t + 1 - a$) holds x_{t+1-a}; that cell is the one under the head at tick $t + 1$. The same argument applies to B. □

The machine is therefore exactly two shift registers, of lengths a and b, whose feedback shares one nonlinear term. Reversibility is visible in the recurrence itself, $x_{t+1-a} = x_{t+1} \oplus x_t y_t$: the oldest symbol is recovered from the newer ones. The harness checks the recurrence, initial conditions included, on 1,000,000 ticks of random 8/13 tapes from $t = 0$, with no violations (E1).

The Harvard wiring in closed form. With the target on A only, x obeys the same recurrence but $y_{t+1} = y_{t+1-b}$: loop B is a periodic drive with period b, and x is a register clocked by it. Homoiconicity is, precisely, the shared term g_t feeding back into *both* registers.

What this explains. moop's primitive observation, MAYBE, steps the core once and returns the cell under loop A's head: one symbol of the x-sequence. It is "unpredictable-looking but replayable" for the same reason the keystream of an invertible NLFSR cipher is. Over 10^6 draws from random 8/13 tapes the share of ones is 0.4976 (E7). Births, which seed a new body from MAYBE draws (§8), are therefore seeded from a keystream.

5 Homoiconicity as the absence of a conserved program

Definition 5 (Structural homoiconicity)**.** A loop is *autonomous* if its next head symbol is a function of its own past alone. A wiring is *homoiconic* if no loop is autonomous: every part of the state that acts on the rest is also acted on by it. An autonomous loop is exactly a "program tape".

By Theorem 4, under moop's wiring each register's update depends on the other through g_t (flip x_t while $y_t = 1$ and y_{t+1} flips), so neither loop is autonomous. Under the Harvard wiring loop B is autonomous. This is the form moop's intended homoiconicity takes at the system layer, where there is no instruction encoding on the tapes; at the user layer it takes the representational form (§9).

Proposition 6 (Harvard conserves its program)**.** *Under the Harvard wiring B is invariant, so the state space splits into 2^b sectors, one per program, each closed under the dynamics. No orbit of the epoch map is longer than $2^a - 1$.*

Proof. B is never a target. Within a sector the epoch map is a bijection on the 2^a values of A, and $A = 0$ is fixed because $x \equiv 0$ forces $g \equiv 0$. □

Proposition 7 (Both loops are written)**.** *Under moop's wiring, every state that is not a fixed point of E fires during its epoch, and every firing flips one cell of each loop.*

Proof. If no tick of an epoch fires, the epoch only rotates the heads, which return; the state is then fixed. A firing XORs 1 into a target on each loop. □

Proposition 7 says neither loop is ever read-only along a non-trivial orbit. A stronger question is whether a loop can be written and yet *restored* at every epoch boundary, so that its epoch-sampled value is conserved along the orbit. That is an empirical question, and we answer it by enumeration.

Census. For nine coprime loop pairs we enumerate every tape state under both wirings and group the states into orbits of E (E2, Table 1).

Three things stand out. First, the Harvard wiring never changes B, by construction, and its orbits are short and numerous: at 8/13, 442,990 orbits, none longer than 255 epochs. Second, the homoiconic wiring merges those sectors into a few long orbits: 8,938 orbits, the longest 506,447 epochs (≈ 52.7 million ticks), and 99.6% of states lie on orbits along which B changes. Third, at these nine sizes *every* non-trivial orbit changes both loops.

loops	wiring	states	orbits	fixed	non-trivial	change A	change B	longest
2/3	moop	32	14	11	3	3	3	11
	Harvard		26	23	3	3	0	3
3/4	moop	128	34	23	11	11	11	31
	Harvard		70	55	15	15	0	7
3/5	moop	256	46	39	7	7	7	109
	Harvard		110	79	31	31	0	7
4/5	moop	512	50	47	3	3	3	233
	Harvard		202	127	75	75	0	15
5/7	moop	4,096	194	159	35	35	35	1,424
	Harvard		1,616	1,097	519	519	0	31
5/8	moop	8,192	380	287	93	93	93	3,053
	Harvard		2,552	1,631	921	921	0	31
7/8	moop	32,768	440	383	57	57	57	9,585
	Harvard		5,046	4,087	959	959	0	127
8/9	moop	131,072	850	767	83	83	83	32,509
	Harvard		16,376	10,859	5,517	5,517	0	255
8/13	moop	2,097,152	8,938	8,551	387	387	387	506,447
	Harvard		442,990	339,427	103,563	103,563	0	255

Table 1: Orbits of the epoch map, every tape state enumerated. "Change A/B" counts the orbits along which that loop's epoch-boundary value changes; "longest" is in epochs. The Harvard bound $2^a - 1$ (Proposition 6) is attained at every size.

A single orbit tours program-space. The longest 8/13 orbit visits 8,191 of the 8,192 possible values of B and 255 of the 256 values of A (E3)—every non-zero "program" and every non-zero "datum". Under the Harvard wiring every orbit visits exactly one program.

Widening the census. The nine sizes were chosen before the question was asked, so we swept every pair $2 \le a \le b$ with $a + b \le 22$: 100 pairs (54 coprime), 78,293,680 states, and 2,053,629 non-trivial orbits.

Observation 8 (Restoring orbits are rare and short)**.** *In 89 of the 100 pairs, every non-trivial orbit changes both loops' epoch-boundary values. In the remaining 11 pairs (eight coprime) there are 184* restoring *orbits, covering 438 states in total, on which one loop returns to the same value at every epoch boundary. None is longer than three epochs. The restored loop is A in seven pairs and B in four (Table 6 in Appendix A).*

By Proposition 7, the restored loop is still written during every epoch of a restoring orbit. On every restoring orbit the gate fires between 4 and 16 times per epoch, always an even number (E2). It must be even: each firing flips one cell of the restored loop, and a loop comes back only if each of its cells is flipped an even number of times. So no loop is ever a read-only program tape, but the orbit-level property is not universal, and we do not claim it is. We have no characterisation of which sizes admit restoring orbits.

Reading. Dynamically, homoiconicity is the removal of a conservation law: the program stops being a constant of motion (Figure 2). Reversibility then guarantees that removing it costs nothing.

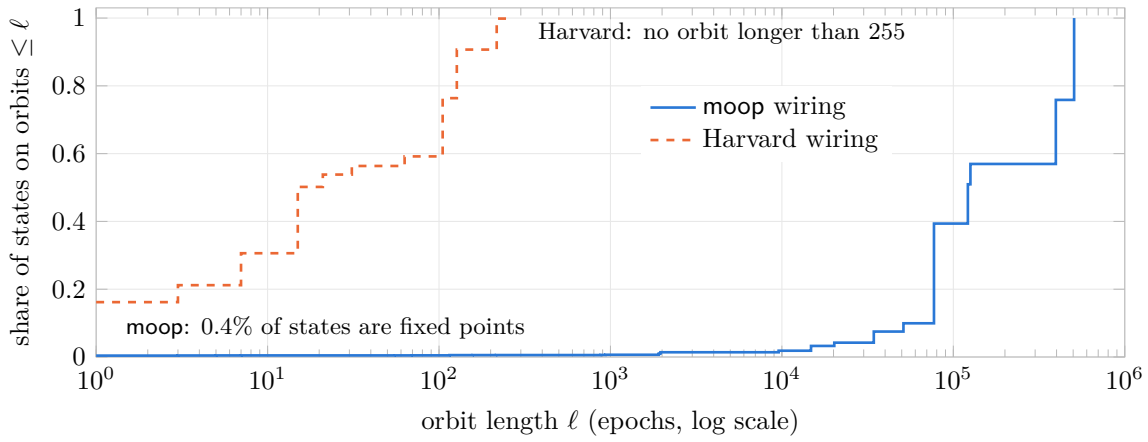


Figure 2: Cumulative distribution of the 8/13 tape states by the length of the orbit they lie on. Under the Harvard wiring every state is on an orbit of at most 255 epochs; under moop's wiring most states lie on a handful of orbits of tens or hundreds of thousands of epochs.

	program conserved (Harvard)	program in mutable state (homoiconic)
irreversible	data history lost; the program never changes	program history can be lost: "self-modification with amnesia"
reversible	data history recoverable; the program never changes	program *and* data history recoverable from any state

Table 2: The design notes' claim as a 2×2. Only the lower-right cell has a program history that is both non-trivial and recoverable. moop's system layer is designed to occupy that cell and its user layer the upper-right one.

6 The synergy, stated precisely

Throughout this section S is a finite state set, $f : S \to S$ is the step function, and a *program* is a projection $\pi : S \to P$ onto a designated part of the state. The program trajectory from s is $P_t = \pi(f^t(s))$. When f is a bijection and all of S is the machine's state, nothing outside the state is needed to go forward or back; we call that *causal closure*.

Proposition 9 (The 2×2)**.** *Let S_0 be uniformly distributed.*

(a) If f is a bijection, $H(S_0 \mid S_t) = 0$ for every t; in particular the program trajectory $P_0, \ldots, P_T$ is a function of any single S_t.

(b) If f merges two states with different programs ($f(s) = f(s')$, $\pi(s) \neq \pi(s')$), then $H(P_0 \mid S_1) > 0$.

(c) Under the Harvard wiring $P_t = P_0$ for all t.

(d) Under moop's wiring, $P_t = B_t$ is non-constant on every non-trivial orbit (Proposition 7), and its epoch-sampled value is non-constant on all but the restoring orbits of Observation 8.

Proof. (a) $S_0 = f^{-t}(S_t)$, and $P_\tau = \pi(f^{\tau - t}(S_t))$. (b) Given $S_1 = f(s)$, which has positive probability, P_0 is $\pi(s)$ or $\pi(s')$, each with positive probability. (c) Proposition 6. (d) As stated. □

None of the four parts is deep; their conjunction is the point. Part (b) says only that a stored-program machine *can* lose program history, not that it must. Only under both properties is

there a program history that is guaranteed to be both non-trivial (homoiconicity) and recoverable (reversibility).

moop sits in two cells of Table 2 by design. Its system-facing code, the tapes, is in the lower right: homoiconic and reversible, hence causally closed. Its user-facing code, the taught chains of §9, is in the upper right: homoiconic, and deliberately forgetful. Corollary 11 below says why the second cell is not a compromise. The Harvard row is the road the original wiring took by accident. Table 2 summarises.

Proposition 10 (No convergence)**.** *If f is a bijection, distinct initial states are never in the same state at the same time. In a reversible Harvard machine $f(p, d) = (p, h_p(d))$ with each h_p a bijection on data, if the data D_0 is uniform and independent of the program P, then $I(P; D_t) = 0$ for all t: the data alone says nothing about which program ran.*

Proof. The first claim is injectivity of f^t. For the second, $\Pr[D_t = d \mid P = p] = \Pr[D_0 = h_p^{-t}(d)] = 1/|D|$ for every p, so D_t is uniform and independent of P. □

The first part needs only reversibility; it is the classical-information form of the fact that reversible programs compute injective functions [1]. The contrast between the wirings is in the second part. A full state of either machine determines its program—trivially so under the Harvard wiring, where the program never changes. But under the Harvard wiring the data never carries any trace of the program, while under moop's wiring it does: one epoch from a uniform 8/13 state, loop A alone carries $I(B_0; A_L) = 0.82$ bits about the initial B ($H(A_L) = 8$, $H(A_L \mid B_0) = 7.18$), against exactly 0 for the Harvard wiring (E9).

Corollary 11 (Answers are observations)**.** *Let a computation's answer be* $\mathrm{ans}(s) = \mathrm{obs}(f^T(s))$ *for some observation* $\mathrm{obs} : S \to V$*. If f is a bijection, then* $\mathrm{ans}(s) = \mathrm{ans}(s')$ *for $s \neq s'$ holds only if* obs *identifies the distinct states $f^T(s) \neq f^T(s')$. Every many-to-one aspect of meaning—two programs with the same answer—is contributed by the observation, never by the dynamics.*

Proof. f^T is injective. □

Corollary 11 needs only reversibility, and it constrains the design rather than dictating it. A reversible core cannot itself compute an answer in the extensional sense, because it never merges states: every many-to-one answer must come from a forgetful map outside it. moop's segregation realises this constraint. Its bridge shape, "reversible effect inside, irreversible observation outside" (§8), places the forgetful map exactly at the boundary. Homoiconicity's part is to make the constraint impossible to sidestep. A Harvard machine can make the identification implicitly, by agreeing to read only the data; when the program is part of the state there is no such convention to hide behind, and the forgetting has to be an explicit observation. The corollary does not say where user programs must run: Bennett-style compute–copy–uncompute [2] runs a whole computation on a reversible core with only the final copy irreversible.

Proposition 12 (Recurrence; program identity is the orbit)**.** *If S is finite and f a bijection, every state lies on a cycle, so every program value P_t recurs. For coprime a, b the tick-level orbits of a core (tapes and heads) correspond one-to-one with the orbits of E, and each is L times as long.*

Proof. A permutation of a finite set decomposes into cycles. For the correspondence: by the Chinese remainder theorem every head pair is $(t \bmod a, -t \bmod b)$ for a unique $t \in \mathbb{Z}_L$, so every core state is reached from a heads-$(0, 0)$ state, and a tick-level orbit passes heads $(0, 0)$ exactly at multiples of L, where it visits an orbit of E. □

Under homoiconicity, then, every self-modification is eventually undone, and "which program is running" is not well defined; only the orbit is invariant. The 8/13 machine has exactly 8,938 computations in this sense. The theorem is about identity, not a mechanism: recurrence can take 52.7 million ticks, and the practical undo is the step-back of Proposition 2.

Self-modification with an audit trail. The practical face of Proposition 9(a) is time-travel debugging of code that rewrites itself. The fault may lie in an instruction that no longer exists, and stepping back recovers it exactly. Omniscient debuggers [17] and record–replay systems [22] achieve this by *logging*; for the core alone the log is unnecessary, because the past is recomputable from the present. Once the user layer feeds observations back into the core, the transcript of observations becomes the log (Theorem 15).

7 Forgetting exactly once: causal pruning

moop's core has one sanctioned way to lose information. Each cell carries a mark; a firing marks its two controls and two targets. The rule records participation in a state change, not counterfactual dependence: a 0 control that prevented a flip is not marked, although on a counterfactual account of causation such as Halpern and Pearl's [11] it would count as a cause. `moop_core_prune` zeroes every unmarked cell, clears the marks, and starts a new epoch; `moop_core_run` prunes at every epoch boundary.

Lemma 13 (Rewind)**.** *Start an epoch at state S_0 with clear marks, run k ticks to S_k, and prune to S'_k. Running k ticks back from S'_k yields S_0 with the pruned cells zeroed: every kept cell retraces its past exactly.*

Proof. Let U be the set of cells unmarked at tick k, and let S'_t be S_t with the cells of U zeroed. We show that S'_t steps to S'_{t+1}. If tick t fires in S_t, its controls and targets are marked, so they are not in U; S'_t has the same controls and fires on the same targets. If tick t does not fire in S_t, then S'_t, whose cells are pointwise $\leq$ those of S_t, does not fire either. In both cases the cells of U are untouched, and they are never targets of a firing in S, so they are constant over $[0, k]$. Hence the trajectories agree off U, and by Proposition 2 stepping back from S'_k retraces $S'_{k-1}, \ldots, S'_0$. □

The harness confirms the lemma in 100,000 random trials, 41,520 of which actually prune something, with no mismatches (E5). The lemma is a statement about the past. It is not the factorisation "web $\otimes$ inert" that the design notes once claimed: counterfactually, a 0 cell that blocked a firing did shape the rest of the state, and a pruned cell might have fired in a later epoch.

Pruning $\times$ homoiconicity. Because code and data share the tapes, an inert cell is dead code and dead data at once, so garbage collection and dead-code elimination are one operation. In its extreme form, 8,447 of the 2^{21} states of the 8/13 machine never fire during an epoch; at the boundary they are pruned to zero entirely.

Proposition 14 (What the marks record)**.** *Within an epoch, stepping back adds no marks. After k forward ticks from the start of an epoch, with no steps back, the marks are a function of S_k and k; with epoch-aligned prunes, k is itself a function of the head positions. After steps back, the marks are those of the furthest tick reached, which the state does not record.*

Proof. Stepping back from S_{t+1} applies R^{-1} and then F at tick t's alignment. The controls are those of forward tick t, because F never writes its controls, so the back-step fires if and only if the forward

density	2%	4%	6%	8%	10%	20%	30%	40%	50%
set bits pruned	82.9	68.4	55.5	44.6	35.7	12.0	3.4	0.9	0.2
runs that prune	30.9	46.1	51.9	50.0	45.5	19.7	6.2	1.5	0.3

Table 3: First-epoch pruning on random 8/13 tapes, 4,000 runs per density (percentages). Fewer runs prune at 2% than at 6% because very sparse tapes often start with nothing set at all.

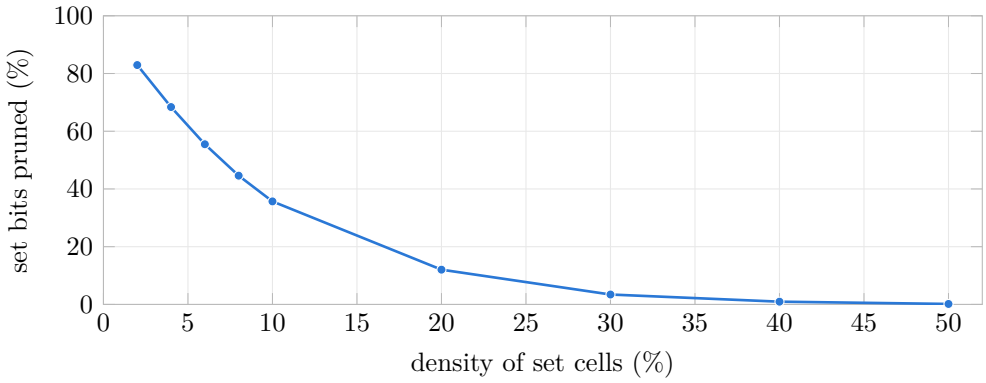


Figure 3: Share of initially set bits pruned at the first epoch boundary, by tape density (8/13 cells, 4,000 runs per point).

tick did, and marks the same four cells. For the second claim, rewind k ticks to S_0 and replay with clear marks. For the third, the heads are $(k \bmod a, -k \bmod b)$, which determine k modulo L by the Chinese remainder theorem (for any a and b, coprime or not). The last claim follows from the first. □

The harness confirms the first two claims in 20,000 trials (E6). The marks are therefore almost, but not quite, a cache. Run forward only, they are recomputable from the state, and under the project's deletion test—a construct is redundant if a uniform rule covers all its uses—pruning would add no state concept of its own. After steps back, though, they also remember how far the epoch got, which nothing else records. Two further qualifications apply. Value deposits (`<->`, §8) mark the cells they change, so the marks also depend on the deposit transcript; birth deposits do not mark, and exchanges carry marks with their bits. And stepping back *past* the start of an epoch marks cells from the previous epoch's firings.

Where pruning matters. Pruning is a sparsity phenomenon. Table 3 and Figure 3 show the share of initially set bits that are still inert after the first epoch of random 8/13 tapes (E4). At 2% density 82.9% of set bits are pruned; at 50% density, 0.2%.

Landauer accounting. The prune's return value counts the set bits it clears, and it destroys at most as many bits of information as there are unmarked cells. That budget is the system layer's *entire* internal erasure—the analogue, in this design, of the checkpoints at which a Bennett-style computation discards its history [4].

The running language does not prune. The interpreter steps bodies through MAYBE and never calls `moop_core_run`, so no `moop` program prunes today. Turning it on would not be a one-line

	system layer	user layer
memory	the tape loops of every body	conventional, overwritable memory
operators	NOT, CNOT, CCNOT, SWAP (self-inverse)	AND, OR, NAND, NOR, XOR, MAYBE
character	never loses information (except pruning)	forgets freely

Table 4: The two layers.

change. A birth draws $a + b$ ticks from its generator, and a prune inside that window would break the generator's ability to rewind to before the birth (§8). When a body may prune without breaking the promises made by births is an open question.

8 Two memories, two logics

moop segregates its machine into two layers (Table 4). Corollary 11 says why answers need a forgetful layer. This section makes precise what may cross between them.

Placement by information behaviour. XOR appears on both sides. As a two-input, one-output function it discards an input and belongs to the user layer; as CNOT it computes the same XOR in place, keeps its control, and belongs to the gate layer. What places an operator is whether its inputs survive, not which Boolean function it computes.

A calculus of crossings. Let $\sigma \in \Sigma$ be the system state (all tapes and heads) and $\rho \in R$ the user state. A *program* is a fixed sequence of instructions of five kinds:

- $\mathsf{sys}(g)$: $\sigma \leftarrow g(\sigma)$, with g a bijection of Σ (a step, a step-back, or an exchange of two bodies' tapes);
- $\mathsf{obs}(\pi, u)$: emit $o = \pi(\sigma)$ to the *transcript*, then $\rho \leftarrow u(\rho, o)$;
- $\mathsf{dep}(q, \{g_v\})$: $\sigma \leftarrow g_{q(\rho)}(\sigma)$, where every g_v is a bijection of Σ (an XOR deposit of the value v);
- $\mathsf{usr}(h)$: $\rho \leftarrow h(\rho)$, with h arbitrary;
- $\mathsf{new}(n)$: append n blank cells to σ.

MAYBE is sys(step) followed by obs of the head cell; the innate `value` message is obs of loop A; `n <-> box` is dep.

Theorem 15 (Transcript)**.** *Run a prune-free program from (σ_0, ρ_0) to (σ_T, ρ_T), producing transcript O.*

(i) ρ_T is a function of ρ_0 and O: the user layer sees the system only through the transcript.
(ii) σ_0 is a function of σ_T, ρ_0, and O.
(iii) If no deposit's value depends on O (open loop), then $\sigma_0 \mapsto \sigma_T$ is injective for each fixed ρ_0.

Proof. (i) By induction: only obs and usr change ρ, and they use only ρ and the emitted o. (ii) By (i), replaying the user layer from ρ_0 and O yields every deposit value v_i. Now invert the system half in reverse order: $\mathsf{sys}(g)$ by g^{-1}, dep by $g_{v_i}^{-1}$, and new by dropping the appended cells, while obs and usr leave σ alone. (iii) The inverse in (ii) then uses no part of O. □

Part (iii)'s hypothesis cannot be dropped. Observing a body's value and depositing it back (a fresh session),

```
moop> box is a world
moop> box -> value
181
```

crossing	moop construct	known as
copy out	`maybe`, `value` (observation)	measurement (QRAM [15])
controlled in	`n <-> box` (XOR deposit)	classically controlled gate
create	blank cells for a birth	*create* [14]; ancilla
erase	epoch prune	*erase* [14]; Landauer

Table 5: Every point at which information crosses into or out of the system layer. A birth is *create*, then copy out, then controlled in.

```
moop> n is box -> value
moop> n <-> box
a proto
moop> box -> value
0
moop> n <-> box
a proto
moop> box -> value
181
```

blanks the body's loop A, because $A \oplus A = 0$, and depositing `n` again restores it. (A `<->` answers with the body it changed, which the REPL prints as `a proto`.) On the 8/13 tapes this closed loop maps all 2^{21} states onto 8,192 (E8). Nothing has been lost yet: `n` still holds the value. The loss happens only when the user layer forgets `n`. This is reset by measurement and feedback, and it is exactly Bennett's account of Maxwell's demon [3]: the measurement is reversible, and the cost is paid when the demon's memory is erased. Here the user layer is the demon's memory.

Corollary 16 (Erasure is pruning or export)**.** *In the calculus extended with pruning—and in moop, to the extent that its C code follows the calculus (§11)—the system layer destroys information without a record only by pruning. Every other loss is an export through observation: given the user layer's initial state, it is recorded in the transcript, and it becomes physical erasure only when the user layer forgets.*

The harness checks Theorem 15(ii) on 100,000 random closed-loop programs of 64 instructions each (2,560,676 observations): the initial tapes were recovered every time (E8). The unit tests include the same check and the REPL reset above.

The crossings. Table 5 maps every crossing onto the vocabulary of §2. The system layer is information-preserving except at these four points. Births are not a fifth: since the changes described in this paper, a birth is literally `new`, then one MAYBE draw per cell from the generator, each deposited onto a blank cell with CNOT. The C code asserts that the child's cells are blank.

In linear-logic terms [9] the core is linear: no step duplicates or discards. A birth is a controlled *contraction* (fresh cells receive copies of observations) and pruning a controlled *weakening*, the "discharge of premises that were never used in any inference" in the project's own phrase.

Actors and protos. moop's objects sit on this boundary. An actor's body is a core and its surface is a table of hosted, irreversible messages. Undo the body's reversible steps and its whole state returns; the replies it gave are the only irreversible product. The irreversibility lives entirely in what was *said*, never in what *is*. Protos add two relations that moop keeps apart. *Generation* (who made you) is the only downward path from system to user layer, and it costs the generator nothing

irreversible: step it back $a+b$ ticks and it returns to the moment before the birth. *Delegation* (whom you defer to) walks parent links among user-facing protos only. A birth starts with its generator as parent, `child <- parent` declares another, and a declaration that would close a loop is refused, so every lineage ends; delegation never crosses the boundary.

9 The synergy at the language surface

moop pairs an Io-like grammar with a Quorum-like vocabulary. There is one grammar rule and four relational operators whose arrow shapes follow information behaviour: `->` sends a message and `<-` declares inheritance, both one-way and forgetful; `<->` is two-way and gate-backed; `is` names. The arrows are canonical, and code prints with them; the words `ask`, `inherits` and `mirrors` are exact aliases. Blocks use pythonic indentation instead of `end`. The indefinite article births: `rex is a dog` introduces a new proto generated from `dog`, while `rex is dog` only aliases it. The vocabulary follows Quorum's evidence about which words novices read well [25]—`is`, the article, and word aliases for the arrows—while the arrows stay canonical because their shapes carry an information behaviour that a word cannot show. The object model follows Self and Io [7, 28] in using prototypes rather than classes, and, as in Io, a proto's parent can be declared.

Code that is its own inverse. Every `<->` form is an involution: `5 <-> box` XOR-deposits 5 onto the body's loop A, and `box <-> other` swaps two bodies' A-loops. Applying either twice is the identity (tested), so for these forms the program text is its own undo.

General functions, without a new form. The design notes list number-to-number bijections under `<->` ("`celsius <-> fahrenheit`") as open. The analysis above reduces half of that problem. The standard reversible embedding [2, 26] turns any function f, reversible or not, into the involution $(x, y) \mapsto (x, y \oplus f(x))$, and moop's deposit already accepts any number on its left. With f the identity it works today, in a fresh session:

```
moop> c is a world
moop> f is a world
moop> c -> value
181
moop> f -> value
52
moop> c -> value <-> f
a proto
moop> f -> value
129
moop> c -> value <-> f
a proto
moop> f -> value
52
```

This is a CNOT copy between bodies ($52 \oplus 181 = 129$), and doing it twice is the identity. For the forward direction, then, no new `<->` form is needed; what is missing is a way to compute f in the user layer—arithmetic or logic operators, and messages that take arguments. The embedding never computes f^{-1}, though, which a two-way reading of `celsius <-> fahrenheit` implies, and `1 <-> 2` is still refused. So the deletion test does not dissolve the whole construct. In-place bijections are the other route: a bijection written as a sequence of involutions is inverted by reversing the sequence—exactly the shape of $\mathsf{F} \circ \mathsf{R}^{-1}$ in Proposition 2. NOT, CNOT and CCNOT build every

even permutation of n-bit values ($n \geq 4$), and every permutation with one extra bit [23]. Reversing a chain is an operation on code as data. moop's reflection (below) makes code a value that can be taken apart, but not yet assembled, so this route still needs a way to join code.

Layered homoiconicity: reversible where system-facing. moop's user code is homoiconic in its own medium: a taught message body is the parser's chain structure, stored and run at each send. Re-teaching a message *replaces* the chain—homoiconicity without reversibility, the upper-right cell of Table 2. That is the design, not a gap: moop reserves reversibility for system-facing code, and by Corollary 11 the user layer is where answers live, and answers require forgetting. The causal closure of §6 therefore holds exactly where it was meant to, in system-facing code. Keeping user programs in the irreversible layer, off the tapes, is consistent with the same corollary but not required by it, since compute–copy–uncompute could run them on a reversible core. The project's rule that nothing user-facing goes on the tapes is a design choice the theorems permit, not one they force.

Reflection: quotes mention code. User-layer homoiconicity is reached through the oldest device written English has for the distinction between using words and mentioning them: quotation marks. A quote is the quoted statement itself, as a value, parsed but never evaluated; quotes nest by alternating double and single marks, as in English prose. In message position a quoted name is mentioned rather than sent, so it designates the receiver's code for that message, found by the same lookup a send uses. With `is` it designates the place where that code lives:

```
moop> dog is a world
moop> dog -> mood is -> maybe
moop> dog -> "mood"
"-> maybe"
moop> cat is a world
moop> cat -> "mood" is dog -> "mood"
moop> cat -> "mood"
"-> maybe"
```

The quote moves across `is`: `x -> m is c` and `x -> "m" is "c"` teach the same thing, so teaching needed no new form. Running a code value needed none either—teach it, then ask—so by the deletion test there is no `eval`. Reads and writes copy, so a code value never aliases a teaching. Code prints back in canonical spelling, which can be typed in again.

Code also comes apart. Every code value answers two innate messages, as every proto answers `maybe` and `value`: `first` is its first part and `rest` is the code that follows it. A block's parts are its lines; a definition's are what it defines and what it is; a chain's are its head term and then its steps:

```
moop> plan is "dog -> mood <-> 5"
moop> plan -> rest
"-> mood <-> 5"
moop> plan -> rest -> first
"-> mood"
moop> plan -> rest -> rest -> rest
"5"
```

Headless chains are what make this work: because a chain may begin with an operator, whatever follows a chain's first part is always well-formed code, so every answer prints, re-enters, and can be taught. An operator is a part but not code on its own, so `first` refuses it. Taking code apart and teaching the pieces is editing by selection. What reflection does not yet provide is assembly: joining two code values needs a binary form or message arguments, which moop does not have for any kind of data yet.

10 Method and reproducibility

All experiments are in one C file, `docs/paper/experiments.c`, linked against the real core; the Harvard control reimplements the original `fire` function. E2, E3, E9 and the sweep are exhaustive and therefore exact. E1 (50 runs of 20,000 ticks, checked from $t = 0$ with the initial conditions of Theorem 4), E4 (4,000 runs per density), E5 (100,000 trials), E6 (20,000), E7 (10^6 draws), and E8 (100,000 programs) use a fixed-seed xorshift generator, so every number reproduces. (The REPL transcripts come from the interpreter and the line count from the source.) The whole harness runs in about a minute and a half on one core and writes the data behind Figures 2 and 3. The repository's unit tests (`make test`) cover the invariants the paper relies on: the round-trip, both loops transformed, the 4/5 orbit census, involutions, the rewind lemma, the marks within an epoch, the transcript recovery, the gate-backed births, and the generator's rewind.

11 Limitations and threats to validity

1. **The language does not execute on the core.** The core is a substrate for objects' bodies, not the processor that runs the program. Section 9 argues this placement is consistent with Corollary 11, but it means moop is not a non-von-Neumann *computer*; it is a language whose objects have reversible bodies.
2. **No universality.** The loops are finite and the wiring is fixed; the project's phrase "Turing tape" is a misnomer for fixed-length loops. A universality result would need unbounded loops and an embedding of a known universal reversible system.
3. **Structural, not instruction-level, homoiconicity.** There is no instruction encoding on the tapes, so our results concern state, not code in the usual sense. At the user layer, code is a value (reflection by quotation) that can be taken apart but not yet assembled from pieces.
4. **The orbit-level property is not universal.** Observation 8 is exhaustive only up to $a + b \leq 22$, for one wiring, and it has exceptions; we have no characterisation of them.
5. **Recurrence is conceptual.** Recurrence times of tens of millions of ticks make Proposition 12 a statement about identity, not an undo mechanism.
6. **Pruning is barely exercised.** It rarely acts on dense tapes (Table 3), the running language never prunes, and a manual mid-epoch prune breaks the recovery of the tick count from the heads.
7. **The user layer is partly nominal.** The interpreter keeps user state in ordinary C structures rather than the RAM module, and of the user logic only MAYBE is reachable from the surface.
8. **Segregation is enforced by convention.** The C modules respect the layer rules by discipline; Theorem 15 holds for the calculus, not for arbitrary C. A type or effect system would enforce it.

12 Related work

Reversible computing. moop's core is a reversible dynamical system in the tradition of conservative logic and reversible cellular automata [8, 21, 27]. Its distinguishing choices are a single gate pair on two counter-rotating loops and the requirement that no loop act as a program. Bennett's analysis of uncomputation [2, 4] is the model for treating pruning as the one, explicit discard. Reversible processors such as Pendulum [29] target hardware efficiency; we make no efficiency claim.

Reversible languages. Janus [18, 31] and its successors make whole programs reversible and invert them syntactically. moop instead keeps a reversible substrate under an irreversible surface. The closest work is James and Sabry's information effects [14], which treat creation and erasure of information as effects layered on a reversible core. moop's crossings map onto theirs (Table 5), adding observation and classically controlled deposits from the QRAM model [15]. Our transcript theorem has the flavour of their results for an untyped, object-based setting. Reversible garbage collection [20] and causal-consistent reversibility [6] are related to pruning and its causal marks.

Homoiconicity and reflection. Lisp [19] and reflective towers [24] are representationally homoiconic, and so is moop's user layer, with quotation in place of Lisp's `quote` and no `eval`. Our structural definition for the system layer is closer to the stored-program concept [30], phrased dynamically: no autonomous program subsystem.

Shift registers. Theorem 4 places the core among coupled NLFSRs, the state machines of hardware-oriented stream ciphers [5, 10, 12]. We use the connection only to explain the behaviour of MAYBE and make no cryptographic claim.

Objects, actors, and syntax. Self [28] and Io [7] are prototype-based; actors [13] inspired moop's non-hereditary actors; and Quorum's empirical work on syntax [25] motivates its word operators.

Debugging. Omniscient debugging [17] and record–replay [22] provide backward execution by logging; causal closure provides it by recomputation.

13 Future work

The most direct next steps are:

- characterise the loop sizes that admit restoring orbits, and prove Observation 8 where it holds;
- expose user-layer computation (logic words, arguments), so that the Bennett-embedding route to general functions becomes usable;
- settle how pruning and generation share a body, then let the running language prune;
- enforce the segregation with a type or effect system that has Theorem 15 as a soundness result;
- add assembly of code from parts (taking it apart exists), enabling inversion by reversal;
- look for universality with unbounded loops or a ring of gates;
- reclaim pruned cells (shrinking the loops) rather than zeroing them in place.

14 Conclusion

Homoiconicity removes the conserved program; reversibility makes the removal lossless. A machine with both properties remembers every program it has been; because it never lets two states converge, it cannot itself say that two computations meant the same thing—only an observation can. moop's two memories put that observation at the boundary: homoiconic everywhere, reversible where it faces the system, and forgetful only where answers are made. Its one internal act of forgetting, causal pruning, removes only what never decided anything, and every other loss is an export that the user layer, like Maxwell's demon, pays for when it forgets.

References

[1] Holger Bock Axelsen and Robert Glück. What do reversible programs compute? In *Foundations of Software Science and Computational Structures (FoSSaCS)*, volume 6604 of *LNCS*, pages 42–56. Springer, 2011. doi: 10.1007/978-3-642-19805-2_4.

[2] Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973.

[3] Charles H. Bennett. The thermodynamics of computation—a review. *International Journal of Theoretical Physics*, 21(12):905–940, 1982. doi: 10.1007/BF02084158.

[4] Charles H. Bennett. Time/space trade-offs for reversible computation. *SIAM Journal on Computing*, 18(4):766–776, 1989.

[5] Christophe De Cannière and Bart Preneel. Trivium. In Matthew Robshaw and Olivier Billet, editors, *New Stream Cipher Designs: The eSTREAM Finalists*, volume 4986 of *LNCS*, pages 244–266. Springer, 2008.

[6] Vincent Danos and Jean Krivine. Reversible communicating systems. In *CONCUR 2004 – Concurrency Theory*, volume 3170 of *LNCS*, pages 292–307. Springer, 2004. doi: 10.1007/978-3-540-28644-8_19.

[7] Steve Dekorte. Io: A small programming language. In *Companion to the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), Dynamic Languages Symposium*, 2005.

[8] Edward Fredkin and Tommaso Toffoli. Conservative logic. *International Journal of Theoretical Physics*, 21(3–4):219–253, 1982.

[9] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–102, 1987.

[10] Solomon W. Golomb. *Shift Register Sequences*. Holden-Day, 1967.

[11] Joseph Y. Halpern and Judea Pearl. Causes and explanations: A structural-model approach. Part I: Causes. *The British Journal for the Philosophy of Science*, 56(4):843–887, 2005.

[12] Martin Hell, Thomas Johansson, and Willi Meier. Grain: A stream cipher for constrained environments. *International Journal of Wireless and Mobile Computing*, 2(1):86–93, 2007. doi: 10.1504/IJWMC.2007.013798.

[13] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.

[14] Roshan P. James and Amr Sabry. Information effects. In *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 73–84, 2012.

[15] Emanuel Knill. Conventions for quantum pseudocode. Technical Report LAUR-96-2724, Los Alamos National Laboratory, 1996.

[16] Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.

[17] Bil Lewis. Debugging backwards in time. In *Proceedings of the Fifth International Workshop on Automated Debugging (AADEBUG)*, 2003. arXiv:cs/0310016.

[18] Christopher Lutz and Howard Derby. Janus: A time-reversible language, 1982. Letter to R. Landauer, California Institute of Technology.

[19] John McCarthy. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4):184–195, 1960.

[20] Torben Ægidius Mogensen. Garbage collection for reversible functional languages. In *Reversible Computation (RC)*, volume 9138 of *LNCS*, pages 79–94. Springer, 2015. doi: 10.1007/978-3-319-20860-2_5.

[21] Kenichi Morita. *Theory of Reversible Computing*. Springer, 2017.

[22] Robert O'Callahan, Chris Jones, Nathan Froyd, Kyle Huey, Albert Noll, and Nimrod Partush. Engineering record and replay for deployability. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 377–389, 2017.

[23] Vivek V. Shende, Aditya K. Prasad, Igor L. Markov, and John P. Hayes. Synthesis of reversible logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 22(6):710–722, 2003.

[24] Brian Cantwell Smith. Reflection and semantics in Lisp. In *Proceedings of the 11th ACM Symposium on Principles of Programming Languages (POPL)*, pages 23–35, 1984.

[25] Andreas Stefik and Susanna Siebert. An empirical investigation into programming language syntax. *ACM Transactions on Computing Education*, 13(4):19:1–19:40, 2013.

[26] Tommaso Toffoli. Reversible computing. In *Automata, Languages and Programming (ICALP)*, volume 85 of *LNCS*, pages 632–644. Springer, 1980.

[27] Tommaso Toffoli and Norman Margolus. *Cellular Automata Machines: A New Environment for Modeling*. MIT Press, 1987.

[28] David Ungar and Randall B. Smith. Self: The power of simplicity. In *Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 227–242, 1987.

[29] Carlin James Vieri. *Reversible Computer Engineering and Architecture*. PhD thesis, Massachusetts Institute of Technology, 1999.

[30] John von Neumann. First draft of a report on the EDVAC, 1945. Moore School of Electrical Engineering, University of Pennsylvania.

[31] Tetsuo Yokoyama and Robert Glück. A reversible programming language and its invertible self-interpreter. In *Proceedings of the ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation (PEPM)*, pages 144–153, 2007. doi: 10.1145/1244381.1244404.

loops	non-trivial orbits	restore A	restore B	states on them	longest (epochs)
5/6	45	0	3	6	2
2/13	257	2	0	6	3
5/11	175	0	11	22	2
2/15	689	8	0	24	3
5/13	225	0	5	10	2
8/10*	1,457	6	0	12	2
2/17	1,601	20	0	60	3
6/14*	2,332	8	0	16	2
2/19	3,915	40	0	120	3
5/16	483	0	24	48	2
10/12*	4,773	57	0	114	2

Table 6: The 11 of 100 loop pairs ($2 \le a \le b$, $a + b \le 22$) that have restoring orbits under moop's wiring (*: lengths not coprime). Totals: 184 orbits on 438 of 78,293,680 states.

A Restoring orbits in the sweep

B A REPL session

Births, teaching, delegation, a declared parent, and `<->` applied twice:

```
moop> animal is a world
moop> animal -> mood is
....>     -> maybe
....>
moop> cat is an animal
moop> cat -> mood is 9
moop> dog is an animal
moop> dog -> mood
false
moop> cat -> mood
9
moop> fish is a world
moop> fish <- cat
a proto
moop> fish -> mood
9
moop> box is a world
moop> box -> value
96
moop> 5 <-> box
a proto
moop> box -> value
101
moop> 5 <-> box
a proto
moop> box -> value
96
```

C The design history as a derivation chain

moop's commit messages narrate its history as a sequence of derivations, each change justified by an axiom, with some constructs later derived away. The table lists the rationale each commit records. Homoiconicity was intended from the start; it was first written down in the same commit that corrected the wiring.

commit	change	rationale recorded
`2231ab2`	one CCNOT on two counter-rotating loops	reversibility: gate plus permutation
`53954f3`	two memories, two logics	segregation of reversible and irreversible operators
`7290131`	causal pruning	one explicit, sanctioned erasure
`745ecae`	symmetric targets on both loops	homoiconicity: no read-only program tape
`6a23e49`	non-hereditary actors	the bridge shape, generalised to objects
`a9405bf`	protos: generation apart from delegation	orthogonality of who-made-you and whom-you-defer-to
`e2309ff`	`<->` as involutions	bijective as a tested fact
`843353e`	`self` removed (headless chains)	deletion test: every context has a receiver
`5c17b4c`	`generate` removed (the article)	deletion test: introduction instantiates
`0dd9521`	births as create + observe + deposit	the bridge shape; deletion test on birth as a crossing
`4ca5af7`	reflection: quotes mention code	ubiquitous homoiconicity; deletion test removes `eval`
`0acdfc6`	code answers `first` and `rest`	homoiconicity: code that cannot be examined is not data
`a119376`	arrows canonical; `<-` declares inheritance	Io-like grammar, Quorum-like vocabulary; who-made-you apart from whom-you-defer-to

Table 7: Selected commits and the rationale each records.